

Future-proof submdspan_mapping?

Document #: P3663R0
Date: 2025-03-14
Project: Programming Language C++
LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- 1 Author
- 2 Revision history
- 3 Abstract
- 4 Motivation
 - 4.1 Summary
 - 4.2 Four categories of slice types
 - 4.3 Why undefined behavior?
 - 4.4 Set of contiguous subrange slices was expanded once and may be again
 - 4.5 Other existing categories could be expanded
 - 4.6 WG21 may add a new slice category
- 5 Possible solution: Make submdspan canonicalize slices
 - 5.1 Summary
 - 5.2 Canonicalization rules
 - 5.3 How to apply canonicalization
 - 5.4 What about precondition checking?
 - 5.5 What about performance?
 - 5.6 Hypothetical performance mitigations
 - 5.7 Implementation
 - 5.8 Wording for this solution
 - 5.8.1 Increment `__cpp_lib_submdspan` feature test macro
 - 5.8.2 Change `[mdspan.sub.helpers]`

- [5.8.3 Requirements of all submdspan_mapping customizations](#)
- [5.8.4 submdspan function template](#)
- [5.9 Why we do not propose this solution](#)
- [6 Other solutions](#)
 - [6.1 Make P2769’s submdspan changes a DR for C++26](#)
- [7 Acknowledgements](#)

§ 1 Author

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

§ 2 Revision history

- Revision 0 to be submitted 2025-03-17

§ 3 Abstract

This proposal apprises WG21 of an issue with `submdspan_mapping`. We do not think the issue warrants a change to C++26. However, we present a solution to be applied to C++26, in case WG21 thinks a change is needed.

Currently, the `submdspan` function can call a `submdspan_mapping` customization with any valid slice type. This means that `submdspan_mapping` customizations may be ill-formed, possibly even without a diagnostic, if future C++ versions expand the set of valid slice types. This will happen if [P2769R3](#) is merged into the Working Draft for C++29, as that would generalize *tuple-like* from a concrete list of types to a concept. It may also happen in the future for other categories of slice types.

The natural way to fix this is with a “canonicalization” approach. First, define a fixed set of “canonical” slice types that represent all valid slices, and a function to map valid slices to their canonical versions. Then, change `submdspan` so it always canonicalizes its input slices, and so it only ever calls `submdspan_mapping` customizations with canonical slices.

We do not propose this as a change for C++26 because we believe that this is something implementations would want to do anyway. The possibility of later generalization of *tuple-like* would naturally lead high-quality C++26 implementations to canonicalize such slice inputs. That would minimize code changes both for users, and for implementations (as the Standard Library currently has five `submdspan_mapping` customizations of its own). Nevertheless, we present a draft of a canonicalization design as an option for WG21 to consider.

§ 4 Motivation

§ 4.1 Summary

The current `submdspan` wording has the following constraint ([\[mdspan.sub.sub\] 3.2](#)).

... the expression `submdspan_mapping(src.mapping(), slices...)` is well-formed when treated as an unevaluated operand.

However, nothing currently requires `submdspan_mapping` to be ill-formed when any of its slices is *not* a valid slice type. Thus, the `submdspan` function can call a `submdspan_mapping` customization with any slice type that is valid for `submdspan`. This means that the following scenario invokes undefined behavior.

1. A user defines a layout mapping `my_layout::mapping` with a C++26 - compatible `submdspan_mapping` customization.
2. A later C++ version adds a new valid slice type `new_slice`.
3. `submdspan` is called with `my_layout::mapping` and a `new_slice` slice.

This is true whether the new slice type is in one of the existing four categories of slice types or is in a new category. It is a scenario that has already occurred and may occur again, as we will explain below.

§ 4.2 Four categories of slice types

The Working Draft has four categories of slice types. Let `S` be an input slice type of `submdspan`.

1. “Full”: `S` is convertible to `full_extent_t`. This means “all of the indices in that extent.”
2. “Contiguous subrange”: `S` models *index-pair-like*`<index_type>`, or `S` is a `strided_slice` specialization whose `stride_type` is *integral-constant-like* with value 1. A contiguous subrange slice represents a contiguous range of indices in that extent. The fact that the indices are contiguous (in other words, that they have stride 1) is known at compile time as a function of the slice’s type alone. The special case of `strided_slice` with compile-time stride 1 is the only way to represent a contiguous index range with run-time offset but compile-time extent. (See [P3355r2](#), which was voted into the Working Draft for C++26 at the 2024 Wrocław meeting.)
3. “Strided”: `S` is a specialization of `strided_slice`, but is not a contiguous subrange. It may represent a noncontiguous set of indices in that extent.
4. “Integer”: `S` is convertible to (the layout mapping’s) `index_type`. This means “fix the slice to view that extent at only that index.” Each integer slice reduces the result’s rank by one.

The Draft uses the term *unit-stride slice* to include both full and contiguous subrange slices.

§ 4.3 Why undefined behavior?

Why would calling a C++26 - compatible `submdspan_mapping` customization with a post - C++26 slice type be undefined behavior? Why wouldn't it just be ill-formed? The problem is that the customization might actually be well-formed for new slice types, but it might do something that `submdspan` doesn't expect. For example, it might violate preconditions of the `submdspan_mapping` customization, or it might produce a `submdspan_mapping_result` that violates postconditions of `submdspan` (e.g., views the wrong elements). This could happen for two reasons.

1. The user has overloaded the customization incorrectly. It works correctly for all C++26 slice types, but incorrectly maps the new slice type to one of the four C++26 cases.
2. The user intends for the customization to be well-formed with invalid slice types. It might have behavior that makes sense for the user, but not for `submdspan`, as in the following example.

```
struct my_tag_slice_t {
    template<size_t k>
    friend constexpr size_t get() const {
        if constexpr (k == 0) {
            return 42;
        }
        else if constexpr (k == 1) {
            return 100;
        }
        else {
            static_assert(false);
        }
    }
};

namespace std {
    struct tuple_size<my_tag_slice_t>
        : integral_constant<size_t, 2> {};
    template<size_t I>
    struct tuple_element<I, my_tag_slice_t> {
        using type = size_t;
    };
} // namespace std

template<class... Slices>
friend constexpr auto
submdspan_mapping(const mapping& src, Slices... slices)
{
    if constexpr ((valid_cpp26_slice<Slices> && ...)) {
        // ... all slices are valid C++26 slices,
        // so do the normal C++26 thing ...
    }
}
```

```

    }
    else if constexpr (is_same_v<Slices, my_tag_slice_t> && ...) {
        // my_tag_slice_t is not a valid C++26 slice,
        // but it has an unambiguous interpretation
        // as a pair of indices.
        return std::string("Hello, I am doing something weird");
    }
    else {
        static_assert(false);
    }
}

```

§ 4.4 Set of contiguous subrange slices was expanded once and may be again

WG21 has already expanded the set of contiguous subrange slice types once, and is likely to do so again. This is because it depends on the exposition-only concept *pair-like*. Adoption of [P2819R2](#) into the Working Draft for C++26 at the 2024 Tokyo meeting made complex a *pair-like* type, and therefore a valid slice type for `submdspan`. [P2769R3](#), currently in LEWG review, proposes generalizing *tuple-like* from a concrete list of types to a concept. That would make user-defined pair types valid slice types.

Before adoption of [P2819R2](#), a user-defined layout mapping’s `submdspan_mapping` customization could determine whether a type `T` is *pair-like* by testing exhaustively whether it is `array<X, 2>` for some `X`, `tuple<X, Y>` or `pair<X, Y>` for some `X` and `Y`, or `ranges::subrange` with `tuple_size` 2. P2819 would break a customization that uses this approach. Similarly, adoption of [P2769R3](#) would break customizations that add `complex<R>` (for some `R`) to this list of types to test exhaustively.

One could argue that any type `T` with `get` findable by argument-dependent lookup (ADL), `tuple_size<T>` of 2, and `tuple_element<I, T>` convertible to `index_type` for all `I` has an unambiguous interpretation as a contiguous subrange. Thus, users should write their customizations generically to that interface. A reasonable way to do that would be to imitate the current wording that uses `get<0>` ([\[mdspan.sub.helpers\] 2.2](#)) and `get<1>` ([\[mdspan.sub.helpers\] 7.2](#)). This is a reasonable thing to do. However, in terms of the Standard, there are three issues with that approach.

1. Standard Library implementers need to assume that users read the Standard as narrowly as possible. C++26 does not forbid calling a `submdspan_mapping` customization with a user-defined pair type, but it also does not restrict its behavior in that case.
2. What if a future C++ version calls any type destructurable into two elements a “pair-like type,” even if it does not have ADL-findable `get`?
3. This approach would not help if other slice categories are expanded, or if a new slice category is added, as the following sections explain.

§ 4.5 Other existing categories could be expanded

The only types currently in the “strided” category are specializations of `strided_slice`. Future versions of C++ might expand this category. In our view, this would have little benefit. Nevertheless, nothing stops WG21 from doing so. For example, C++29 might (hypothetically) make a strided slice any type that meets the following exposition-only *strided-slice*<index_type> concept.

```
template<class T>
concept integral_not_bool = std::is_integral_v<T> &&
    ! std::is_same_v<T, bool>;

template<class IndexType, class T>
concept strided-slice = semiregular<T> &&
    convertible_to<T::offset_type, IndexType> &&
    convertible_to<T::extent_type, IndexType> &&
    convertible_to<T::stride_type, IndexType> &&
    requires(T t) {
        { t.offset } -> convertible_to<IndexType>;
        { t.extent } -> convertible_to<IndexType>;
        { t.stride } -> convertible_to<IndexType>;
    };

```

The following user-defined type `my_slice` would meet *strided-slice*<size_t>, even though

- it is not convertible to or from `strided_slice<size_t, size_t, size_t>`;
- it has a base class, which `strided_slice` specializations cannot, per [mdspan.sub.strided.slice] 2; and
- it has members other than `offset`, `extent`, and `stride`, which `strided_slice` cannot, per [mdspan.sub.strided.slice] 2.

```
struct my_base {};

struct my_slice : public my_base {
    using offset_type = size_t;
    using extent_type = size_t;
    using stride_type = size_t;

    offset_type offset;
    extent_type extent;
    stride_type stride;

    // User-declared constructor makes this not an aggregate
    my_slice(extent_type ext) : offset(0), extent(ext), stride(1) {}

    // strided_slice is not allowed to have extra members
    std::string label = "my favorite slice label";
    std::vector<int, 3> member_function() const { return {1, 2, 3};
}

```

```
// my_slice is not convertible to or from strided_slice
template<class Offset, class Extent, class Stride>
my_slice(strided_slice<Offset, Extent, Stride>) = delete;

template<class Offset, class Extent, class Stride>
operator strided_slice<Offset, Extent, Stride>() const = delete;
};
```

Now suppose that a user has defined a `submdspan_mapping` customization in a C++26-compatible way for their custom layout mapping `custom_layout::mapping`. Then, `my_slice` likely would not work for this layout mapping in C++29, even though the hypothetical `_strided-slice_` concept makes it possible to interpret `my_slice` unambiguously as a strided slice, and to write generic code that does so.

Unlike with contiguous subrange slices, users have no way to anticipate all possible ways that future Standard versions might expand the set of valid strided slice types. For example, C++29 could decide that strided slices must be aggregate types (which would exclude the above `my_base`), or that they also include any *tuple-like* type with 3 members all convertible to `index_type` (not what we prefer, but WG21 has the freedom to do this). Furthermore, `my_slice` with `stride = cw<1, size_t>` should reasonably count as a contiguous subrange slice type; expanding one set risks conflicts between sets.

§ 4.6 WG21 may add a new slice category

The same problem would arise if future versions of C++ introduce a new category of slices. For example, suppose that C++29 hypothetically adds an “indirect slice” category that takes an arbitrary tuple of indices to include in that extent of the slice. The Mandates of `submdspan` ([`mdspan.sub.sub`] 4.3) make it ill-formed to call with a new kind of slice, so this hypothetical C++29 addition would need to add a new clause there. That would not be a breaking change in itself. The problem is that nothing currently requires `submdspan_mapping` to be ill-formed when any of its slices is not a valid slice type.

§ 5 Possible solution: Make `submdspan` canonicalize slices

§ 5.1 Summary

There is a tension between `submdspan` users and custom mapping authors. Users of `submdspan` want it to work with all kinds of slice types. For example, they might want to write their own tuple types that are standard-layout if all their template parameters are (unlike `std::tuple`), and would expect `submdspan` to work with them. In contrast, users who customize `submdspan_mapping` for their own layout mappings want a “future-proof” interface. They only want `submdspan` to call `submdspan_mapping` for a known and unchanging set of types.

One solution to this tension is to change `submdspan` so that it “canonicalizes” its slice inputs before passing them to `submdspan_mapping`. The `submdspan` function still takes any valid slice types, but it maps each input slice type to a “canonical” slice type according to rules that we will explain below. As a result, a `submdspan_mapping` customization would ever be called with the following concrete list of slice types.

1. `index_type` (that is, the layout mapping’s `index_type`)
2. `constant_wrapper<Value, index_type>` for some `Value`
3. `strided_slice` where each member is either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`
4. `full_extent_t`

The `constant_wrapper` class template comes from [P2187R7](#), which LEWG design-approved (with design changes not relevant to this proposal) and forwarded to LWG on 2025/03/11 for C++26.

This solution would make it always ill-formed to call an existing `submdspan_mapping` customization with a new kind of slice. With the status quo, calling an existing `submdspan_mapping` customization with a new kind of slice would be ill-formed *at best*, and could be a precondition violation at worst, because nothing prevents `submdspan` from ever calling existing `submdspan_mapping` customizations with new slice types.

§ 5.2 Canonicalization rules

Here are the canonicalization rules for an input slice `s` of type `S` and a input mapping whose extents type has index type `index_type`. The result depends only on the input mapping’s extents type and the slices.

1. If `S` is convertible to `index_type`, then *canonical-ice*`<index_type>(s)`;
2. else, if `S` models *index-pair-like*`<index_type>`, then
`strided_slice{.offset=canonical-ice<index_type>(get<0>(s)),`
`.extent=subtract-ice<index_type>(get<1>(s), get<0>(s)),`
`.stride=cw<index_type(1)>}`.
3. else, if `S` is convertible to `full_extent_t`, then `full_extent_t`;
4. else, if `S` is a specialization of `strided_slice`, then
`strided_slice{.offset=canonical-ice<index_type>(s.offset),`
`.extent=canonical-ice<index_type>(s.extent), .stride=canonical-`
`ice<index_type>(s.stride)}`.

The two exposition-only functions *canonical-ice* and *subtract-ice* preserve “compile-time-ness” of any *integral-constant-like* arguments, and force all inputs to either `index_type` or `constant_wrapper<Value, index_type>` for some value `Value`. The `cw`

variable template comes from [P2187R7](#). Rule (1) mimics the behavior of the exposition-only function `first_` ([\[mdspan.sub.helpers\] 1 - 4](#)).

§ 5.3 How to apply canonicalization

The function `submdspan_canonicalize_slices` would canonicalize all the slices at once. It takes as arguments the input mapping's extents and the input slices of `submdspan_mapping`. Canonicalization would make `submdspan` equivalent to the following code.

```
auto [...canonicalize_slices] =
    submdspan_canonicalize_slices(src.extents(), slices...);
auto sub_map_result =
    submdspan_mapping(src.mapping(), canonicalize_slices...);
return mdspan(src.accessor().offset(src.data(),
sub_map_result.offset),
               sub_map_result.mapping,
               AccessorPolicy::offset_policy(src.accessor()));
```

The `auto [...canonicalize_slices]` bit of code above uses the “structured bindings can introduce a pack” feature introduced by [P1061R10](#) into C++26. The `submdspan_canonicalize_slices` function takes the extents for two reasons.

1. A slice's canonical type depends on the extents' `index_type`.
2. Knowing the extents lets it enforce preconditions on the slices, so that the resulting canonical slices don't need to be checked again. This is why canonicalization is not just a function to be called on each slice without considering its corresponding input extent.

§ 5.4 What about precondition checking?

The `submdspan` function imposes preconditions on both its slice arguments and on the result of `submdspan_mapping` ([\[mdspan.sub.sub\] 5](#)). The preconditions on `submdspan_mapping`'s result require that the user's customization give a mapping that views exactly those elements of the input `mdspan` that are specified by the slices. That is, the customization has to be correct, given valid input slices. Enforcement of this is the user's responsibility.

Enforcement of preconditions on the input slices is the implementation's responsibility. This depends only on the input slices and the input mapping's extents. Just as `mdspan` can check bounds for the input of the `at` function using the input and its extents, `submdspan` can enforce all preconditions on the slices using the input mapping's extents. The current wording expresses these preconditions identically in two places:

1. on the slice arguments of `submdspan_extents` ([\[mdspan.sub.extents\] 3](#)), and
2. on the slice arguments of `submdspan` ([\[mdspan.sub.sub\] 5](#)).

Passing the slices through `submdspan_extents` is not enough to ensure that `submdspan_mapping`'s slice arguments do not violate its (unstated) preconditions. This is because deciding the result mapping's type and constructing it may depend on the slices, not just the result's extents. Another way to say this is that different `slices...` arguments might result in the same extents, but a different layout mapping. For example, if `src` is a rank-1 `layout_left` mapping with extents `dims<1>{10}`, both `full_extent` and `strided_slice{.offset=0, .extent=src.extent(0), .stride=1}` would result in extents `dims<1>{10}`, but different layouts (`layout_left` and `layout_stride`, respectively). This means that slice canonicalization is also an opportunity to avoid duplication of precondition-checking code.

There are two kinds of preconditions on slices:

1. that all indices represented by the slice can be represented as values of type `extents::index_type`, that is, that casting them to `extents::index_type` does not cause integer overflow; and
2. that all indices represented by slice `k` are in `[0, src.extent(k) )`.

The current wording expresses both preconditions via the exposition-only functions `first_` and `last_`. The specification of each of these uses `extents::index-cast` to preprocess their return values (see [\[mdspan.sub.helpers 4\]](#) and [\[mdspan.sub.helpers 9\]](#)). The `index-cast` exposition-only function [\[mdspan.extents.expo\] 9](#) is a hook to enforce the “no overflow” precondition. The “in the extent’s range” precondition is currently up to `submdspan` implementations to check.

Slice canonicalization needs to have at least the “no overflow” precondition, because any integer results are either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`. That is, it always casts input to `index_type`. We propose applying the “in the extent’s range” precondition to slice canonicalization as well. This would let `submdspan` implementations put all their precondition checking (if any) in the canonicalization function.

§ 5.5 What about performance?

We have not measured the performance effects of this approach. Everything we write here is speculation.

In terms of compile-time performance, canonicalization would minimize the set of instantiations of both `submdspan_mapping` and `submdspan_extents` (which customizations of `submdspan_mapping` may call).

In terms of run-time performance, customizations would likely need to copy slices into their canonical forms. This would introduce new local variables. Slices tend to be simple, and are made of combinations of zero to three integers and empty types (like `constant_wrapper` or `full_extent_t`). Thus, it should be easy for compilers to optimize away all the extra types. On the other hand, if they can’t, then the compiler may need to

reserve extra hardware resources (like registers and stack space) for the extra local variables. This *may* affect performance, especially in tight loops. Mitigations generally entail creating subview layout mappings by hand.

The proposed set of canonicalization rules would require turning every *pair-like* type into a `strided_slice`. Pair-like slices are common enough in practice that we may want to optimize specially for them; see below.

§ 5.6 Hypothetical performance mitigations

Here are some options that might mitigate some performance concerns. All such concerns are hypothetical without actual performance measurements.

1. Let Standard layout mappings accept arbitrary slice types. Restrict the proposed changes to user-defined layout mappings. This would optimize for the common case of Standard layout mappings. On the other hand, performing canonicalization for all `submdspan_mapping` customizations (including the Standard ones) would simplify implementations, especially for checking preconditions.
2. Expand the canonicalization rules so that `pair` and `tuple` slices are passed through, instead of being transformed into `strided_slice`. This would optimize a common case, at the cost of making `submdspan_mapping` customizations handle more slice types. That would complicate the code and possibly increase compile-time cost, thus taking away at least some of the benefits of canonicalization.

§ 5.7 Implementation

[This Compiler Explorer link](#) offers a brief and hasty implementation of this solution.

§ 5.8 Wording for this solution

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

Assume that [P2187R7](#) plus fixes from LEWG on its 2025/03/11 review have been applied to the Working Draft. (EDITORIAL NOTE: This makes `constant_wrapper` and `cw` available to the rest of the Standard Library.)

§ 5.8.1 Increment `__cpp_lib_submdspan` feature test macro

In `[version.syn]`, increase the value of the `__cpp_lib_submdspan` macro by replacing `YYMMML` below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_submdspan YYYYMMML // also in <mdspan>
```

§ 5.8.2 Change `[mdspan.sub.helpers]`

Change [mdspan.sub.helpers] as follows.

```
template<class T>
constexpr T de-ice(T val) { return val; }
template<integral-constant-like T>
constexpr auto de-ice(T) { return T::value; }

template<class IndexType, integral-constant-like S>
constexpr constant_wrapper<S::value, IndexType> canonical-ice(S s)
{
    return {};
}

template<class IndexType, class S>
constexpr IndexType canonical-ice(S s) {
    return s;
}

template<class IndexType, class X, class Y>
constexpr auto subtract-ice(X x, Y y) {
    if constexpr (integral-constant-like<X> && integral-constant-
like<Y>) {
        return cw<IndexType>(de-ice(y) - de-ice(x));
    }
    else {
        return IndexType(y - x);
    }
}

template<class IndexType, size_t k, class... SliceSpecifiers>
constexpr IndexType first_(SliceSpecifiers... slices);
```

- ¹ *Mandates:* IndexType is a signed or unsigned integer type.

[Editorial note: Skip down to the end of the section – end note]

```
template<class IndexType, size_t... Extents, class... Slices>
constexpr auto submdspan_canonicalize_slices(
    const extents<IndexType, Extents...>& src, Slices... slices);
```

- ¹² *Constraints:* sizeof...(slices) equals sizeof...(Extents).

- ¹³ *Mandates:* For each rank index k of src, exactly one of the following is true:

- (13.1) — S_k models convertible_to<IndexType>,
(13.2) — S_k models index-pair-like<IndexType>,
(13.3) — is_convertible_v< S_k , full_extent_t> is true, or
(13.4) — S_k is a specialization of strided_slice.

- ¹⁴ *Preconditions:* For each rank index k of src, all of the following are true.

- (14.1) — if S_k is a specialization of `strided_slice`
- (14.1.1) — $s_k.\text{extent} = 0$, or
- (14.1.2) — $s_k.\text{stride} > 0$
- (14.2) — $0 \leq \text{first_}<\text{IndexType}, k>(\text{slices} \dots) \leq \text{last_}<k>(\text{src}, \text{slices} \dots) \leq \text{src}.\text{extent}(k)$

[*Editorial note:* These are the same as the preconditions on the slice inputs of `submdspan_extents` and `submdspan`. They must be imposed here, before conversion to `IndexType` or `constant_wrapper<IndexType, Value>`, in order to include the precondition that forbids integer overflow. – *end note*]

15 *Returns:* a tuple of `src.rank()` elements, where for each rank index k of `src`, the following specifies element k of the return value:

- (15.1) — If S_k models `convertible_to<IndexType>`, then `canonical-ice<IndexType>(s)`;
- (15.2) — else, if S_k models `index-pair-like<IndexType>`, then
`strided_slice{.offset=canonical-ice<IndexType>(get<0>(s)),`
`.extent=subtract-ice<IndexType>(get<1>(s), get<0>(s)),`
`.stride=cw<IndexType(1)>};`
- (15.3) — else, if `is_convertible_v< $S_k$ , full_extent_t>` is true, then `full_extent_t`;
- (15.4) — else, if S_k is a specialization of `strided_slice`, then
`strided_slice{.offset=canonical-ice<IndexType>(s.offset),`
`.extent=canonical-ice<IndexType>(s.extent), .stride=canonical-`
`ice<IndexType>(s.stride)}.`

§ 5.8.3 Requirements of all `submdspan_mapping` customizations

Change `[mdspan.sub.map.common]` as follows.

1 The following elements apply to all functions ~~in `[mdspan.sub.map]`~~ named `submdspan_mapping` that would be found by overload resolution when called with a layout mapping `m` as their first argument and a parameter pack `slices` as their remaining arguments(s).

2 *Constraints:* `sizeof...(slices)` equals `extents_type::rank()`.

3 *Mandates:* For each rank index k of `extents()`, exactly one of the following is true:

- (3.1) — S_k ~~models `convertible_to<index_type>`~~ is `index_type`,
- (3.2) — S_k ~~models `index-pair-like<index_type>`~~ is `constant_wrapper<Value, index_type>` for some value `Value` of type `index_type` such that $0 \leq \text{Value} <$

extents().extent(0),

(3.3) — ~~is_convertible_v<S_k, full_extent_t> is true~~ S_k is full_extent_t, or

(3.4) — S_k is a specialization of strided_slice where each member is either index_type or constant_wrapper<Value, index_type> for some value Value of type extents_type::index_type such that 0 ≤ Value < extents().extent(0).

4 *Preconditions:* For each rank index k of extents(), all of the following are true:

§ 5.8.4 submdspan function template

Change [mdspan.sub.sub] (“submdspan function template”) as follows.

1 Let index_type be typename Extents::index_type.

2 Let canonical_slices be the pack resulting from the following statement.

```
auto [...canonical_slices] =  
submdspan_canonicalize_slices(src.extents(), slices...).
```

3 Let sub_map_offset be the result of submdspan_mapping(src.mapping(),
~~slices~~canonical_slices ...).

[Note 1: This invocation of submdspan_mapping selects a function call via overload resolution on a candidate set that includes the lookup set found by argument-dependent lookup ([basic.lookup.argdep]). — end note]

4 *Constraints:*

(4.1) — sizeof...(slices) equals Extents::rank(), and

(4.2) — the expression submdspan_mapping(src.mapping(), ~~slices~~canonical_slices ...) is well-formed when treated as an unevaluated operand.

5 *Mandates:*

(5.1) — decltype(submdspan_mapping(src.mapping(), ~~slices~~canonical_slices ...)) is a specialization of submdspan_mapping_result.

(5.2) — is_same_v<remove_cvref_t<decltype(sub_map_offset.mapping.extents())>, decltype(submdspan_extents(src.mapping(), ~~slices~~canonical_slices ...))> is true.

(5.3) — For each rank index k of src.extents(), exactly one of the following is true:

(5.3.1) — S_k models convertible_to<index_type>,

(5.3.2) — S_k models index-pair-like<index_type>,

(5.3.3) — `is_convertible_v<Sk, full_extent_t>` is true, or

(5.3.4) — `Sk` is a specialization of `strided_slice`.

6 *Preconditions:*

(6.1) — For each rank index `k` of `src.extents()`, all of the following are true:

(6.1.1) — if `Sk` is a specialization of `strided_slice`

(6.1.1.1) — `sk.extent` = 0, or

(6.1.1.2) — `sk.stride` > 0

(6.1.2) — `0 ≤ first_<index_type,k>( slicescanonical_slices ) ≤ last_<k>(src.extents(), slicescanonical_slices ...)` ≤ `src.extent(k)`

(6.2) — `sub_map_offset.mapping.extents() == submdspan_extents(src.mapping(), slicescanonical_slices ...)` is true; and

(6.3) — for each integer pack `I` which is a multidimensional index in `sub_map_offset.mapping.extents()`,

```
sub_map_offset.mapping(I...) + sub_map_offset.offset ==
src.mapping()(src-indices(array{I...}, slices...))
```

```
sub_map_offset.mapping(I...) + sub_map_offset.offset ==
src.mapping()(src-indices(array{I...}, canonical_slices...))
```

is true.

7 *Effects:* Equivalent to:

```
auto sub_map_result = submdspan_mapping(src.mapping(), slices...);
```

```
auto [...canonicalize_slices] =
    submdspan_canonicalize_slices(src.extents(), slices...);
auto sub_map_result =
    submdspan_mapping(src.mapping(), canonicalize_slices...);
```

```
return mdspan(src.accessor().offset(src.data(),
sub_map_result.offset),
              sub_map_result.mapping,
              AccessorPolicy::offset_policy(src.accessor()));
```

§ 5.9 Why we do not propose this solution

We think that canonicalization is the expected implementation approach anyway, so there is no need to standardize it. Suppose that P2769 is adopted into some future C++ version. Implementations might thus expose users to an intermediate period in which they implement C++26 but not C++29. During this period, users might write `submdspan_mapping` customizations to the narrower definition of *pair-like*. For

example, users might write `std::get` explicitly instead of using ADL-findable `get`, which would only work with Standard Library types that have overloaded `std::get`. This would, in turn, encourage implementers of `submdspan` to “canonicalize” *pair-like* slice arguments (e.g., into `pair` or `tuple`) before calling `submdspan_mapping` customizations, as that would maximize backwards compatibility.

In summary, the possibility of later generalization of *pair-like* would naturally lead high-quality implementations to the desired outcome. That would minimize code changes both for users, and for implementations (as the Standard Library currently has five `submdspan_mapping` customizations of its own). Forcing canonicalization for C++26 would merely anticipate the expected implementation approach.

§ 6 Other solutions

§ 6.1 Make P2769’s `submdspan` changes a DR for C++26

If P2769 is on track to be adopted into some future C++ version, then we could lobby WG21 to treat P2769’s proposed changes to `submdspan`’s wording as a Defect Report (DR) for C++26. This would require no effort prior to acceptance of C++26. However, implementations might expose users to an intermediate period in which they implement C++26 but not the DR. This would have the same practical effect on users as if we had applied the change to C++29 but not as a DR.

§ 7 Acknowledgements

Thanks to Tomasz Kamiński for pointing out this issue.